

Building and Securing a REST API

Building and Securing a REST API — Project Report

Team: Team 1

Date: 30/05/2026

Team members:

1. Aduk Mathiang Ngut Nyang
2. Ntwali Beni David
3. Bol David Garang Dau
4. Ojudun Ayomide Oluwatimilehin

1. Introduction to API Security

REST APIs sit between users and the data they care about. Anyone with the URL can send requests, and without protection, anyone can also read or change that data. This is why API security is treated as a core part of building any API, not an extra step at the end.

For this project, the API exposes mobile money (MoMo) transaction data parsed from SMS messages. That kind of data is sensitive: it includes amounts, sender and receiver identifiers, and timestamps that together describe a person's financial activity. Even though phone numbers and account numbers are masked before storage, the rest of the record is still revealing. An open endpoint would let anyone list every transaction, modify them, or delete them entirely.

Common threats an API has to guard against

- Unauthorized access — anyone hitting an endpoint without credentials.
- Credential theft — attackers obtaining valid usernames and passwords.
- Eavesdropping — credentials or data being read in transit on an insecure network.
- Injection attacks — malicious input that tries to break out of the intended request.
- Replay attacks — valid requests being captured and re-sent by an attacker.

What this API does

The MoMo API uses HTTP Basic Authentication to require credentials on every request. If the credentials are missing or wrong, the server returns 401 Unauthorized and refuses to process the request. This is a simple but real layer of protection: without it, every endpoint would be open to the public. The limitations of Basic Authentication and stronger alternatives are discussed later in this report.

2. API Endpoints Documentation

The API exposes five endpoints covering the standard CRUD operations on transactions. Every endpoint requires Basic Authentication.

Base URL

http://localhost:8000

Authentication

All requests must include an Authorization header containing base64-encoded credentials. With curl, the

-u flag handles the encoding automatically:

```
curl -u admin:password http://localhost:8000/transactions
```

Requests without valid credentials receive a 401 Unauthorized response.

Endpoints at a glance

Method	Endpoint	Purpose
GET	/transactions	List all transactions
GET	/transactions/{id}	Retrieve a single transaction
POST	/transactions	Create a new transaction
PUT	/transactions/{id}	Update an existing transaction
DELETE	/transactions/{id}	Delete a transaction

GET /transactions

GET	/transactions Returns every transaction in the system.
------------	--

Example request

```
➜ dsa git:(main) X curl -u admin:password123 http://localhost:8000/transactions
```

Responses

Status	Description
200 OK	Array of transaction objects.
401 Unauthorized	Missing or invalid credentials.

GET /transactions/{id}

GET	/transactions/{id} Returns a single transaction by its ID.
------------	--

Example request

```
➜ dsa git:(main) X curl -u admin:password123 http://localhost:8000/transactions/1
```

Responses

Status	Description
200 OK	The transaction object.
400 Bad Request	The ID in the URL is not a valid number.
401 Unauthorized	Missing or invalid credentials.
404 Not Found	No transaction exists with that ID.

POST /transactions

POST	/transactions Creates a new transaction. Only four fields are required.
-------------	---

Required fields

Field	Type	Description
amount	number	Transaction amount in RWF
category	string	Category label, e.g. 'Money Transfer'
sender	string	Sender's name
receiver	string	Receiver's name

Example request

```
o → dsa git:(main) X curl -u admin:password123 -X POST http://localhost:8000/transactions \
  -H "Content-Type: application/json" \
  -d '{"amount": 5000, "category": "Money Transfer", "sender": "John Doe", "receiver": "Jane Smith"}'
```

Responses

Status	Description
201 Created	Transaction created. Full transaction object returned.
400 Bad Request	Missing required fields or invalid JSON body.
Status	Description
401 Unauthorized	Missing or invalid credentials.

PUT /transactions/{id}

PUT	/transactions/{id} Partially updates an existing transaction. Only the fields included in the body are changed.
------------	---

Example request

```
➜ dsa git:(main) X curl -u admin:password123 -X PUT http://localhost:8000/transactions/1692 \
  -H "Content-Type: application/json" \
  -d '{"amount": 9999}'
```

Responses

Status	Description
200 OK	Transaction updated. Full updated object returned.
400 Bad Request	Invalid ID format or invalid JSON body.
401 Unauthorized	Missing or invalid credentials.
404 Not Found	No transaction exists with that ID.

DELETE /transactions/{id}

DELETE	/transactions/{id} Removes a transaction from the system.
---------------	---

Example request

```
➜ dsa git:(main) X curl -u admin:password123 -X DELETE http://localhost:8000/transactions/1692
```

Responses

Status	Description
200 OK	Transaction deleted.
400 Bad Request	The ID in the URL is not a valid number.
401 Unauthorized	Missing or invalid credentials.
404 Not Found	No transaction exists with that ID.

Error response format

All error responses follow the same shape:

```
{
  "error": "Route not found"
}
```

3. Testing and Validation

Every endpoint was tested with curl. Both successful and failure cases were captured to confirm that authentication works as expected and that the server returns the correct status codes. The terminal log below shows the sequence of requests that were run during testing.

Server log from the test run

```
[GET] /transactions -> 200
[GET] /transactions -> 401
[POST] /transactions -> 201
[POST] /transactions -> 401
[GET] /transactions -> 200
[PUT] /transactions/1692 -> 200
[PUT] /transactions/1692 -> 401
[DELETE] /transactions/1692 -> 200
[DELETE] /transactions/1692 -> 401
```

Each test case is shown in detail below with a screenshot of the request and the server's response.

3.1 Successful GET with authentication

Listing all transactions with valid credentials returns 200 OK and the full list.

```
[GET] /transactions -> 200
```

3.2 Unauthorized GET with wrong credentials

The same request with invalid credentials returns 401 Unauthorized.

```
[GET] /transactions -> 401
```

3.3 Successful POST

Creating a new transaction with valid credentials and a complete body returns 201 Created.

```
[POST] /transactions -> 201
```

3.4 Unauthorized POST

Sending the same POST without valid credentials returns 401 Unauthorized.

```
[POST] /transactions -> 401
```

3.5 Successful PUT

Updating an existing transaction returns 200 OK and the updated record.

```
[PUT] /transactions/1692 -> 200
```

3.6 Unauthorized PUT

The same PUT request without credentials returns 401 Unauthorized.

```
[PUT] /transactions/1692 -> 401
```

3.7 Successful DELETE

Deleting an existing transaction returns 200 OK and a confirmation message.

```
[DELETE] /transactions/1692 -> 200
```

3.8 Unauthorized DELETE

The same DELETE without credentials returns 401 Unauthorized.

```
[DELETE] /transactions/1692 -> 401
```

3.9 Route not found

Hitting a misspelled or non-existent route (for example /transaction instead of /transactions) returns 404 Not Found.

```
[GET] /transactions/100000 -> 404
```

4. Data Structures and Algorithms — Comparison Results

The assignment required us to look up a transaction by its ID using two different methods and compare how fast they are. We implemented both, ran them against the parsed transaction set, and timed them.

Method 1: Linear Search

Linear search walks the list of transactions one item at a time, comparing each transaction's ID against the target. If it finds a match it returns that transaction; otherwise it returns None. This is an $O(n)$ algorithm — in the worst case, the time it takes grows in direct proportion to the number of transactions.

Method 2: Dictionary Lookup

Dictionary lookup stores the transactions in a Python dictionary keyed by transaction ID. Finding a transaction is just `dictionary.get(id)`. Python uses a hash table internally, so the average time for a lookup is $O(1)$ — the time stays roughly the same no matter how many transactions are stored.

Test setup

- Dataset: 1,691 transactions parsed from the XML file.
- Number of IDs tested: 26 (a mix of existing IDs spread across the dataset, plus one ID that does not exist to test the worst case for linear search).
- Repeats: each ID was looked up 100 times to get measurable timings.
- Timer used: `time.perf_counter()` for high-resolution measurement.

Results

Method	Total time	Relative speed
Linear search	0.037080 seconds	1x (baseline)
Dictionary lookup	0.000132 seconds	280.4x faster

Dictionary lookup completed the same workload roughly 280 times faster than linear search. The difference would grow even larger with a bigger dataset: for a million records, dictionary lookup would still take the same fraction of a millisecond, while linear search would take hundreds of times longer than it did here.

Why dictionary lookup is faster

A Python dictionary is implemented as a hash table. When you ask for transactions[5], Python computes a hash of the key 5 and jumps straight to the location in memory where that entry is stored. It does not look at any other entries. Linear search has no such shortcut: it has to check each transaction in order until it finds the target or reaches the end of the list. The difference is constant-time vs. linear-time, which is the gap the numbers above are showing.

Could we do better?

Dictionary lookup is already very fast for exact-match queries by ID. But there are other search problems where different data structures would perform better:

- Binary search on a sorted array — $O(\log n)$. Slower than a hash table for exact match, but it supports range queries like 'find all transactions with ID between 100 and 200', which a hash dict cannot do efficiently.
- Binary Search Tree (BST) — also $O(\log n)$ average, and like sorted arrays, it supports ordered traversal and range queries.
- B-tree — what most relational databases use internally for indexes. B-trees keep data on disk efficiently by minimizing the number of disk reads per lookup.
- Trie — useful if we wanted to search by prefix on string fields, for example 'all transactions whose sender name starts with Jane'.

5. Reflection on Basic Authentication

Basic Authentication does the job for this assignment, but it is not considered a strong solution for production systems. Below are the main reasons why, followed by stronger alternatives.

Why Basic Auth is weak

Credentials are not encrypted, only encoded

The Authorization header contains username:password run through base64. Anyone who intercepts the request can reverse the base64 in seconds and read the credentials in plain text. Base64 is encoding, not encryption — it provides no secrecy.

Credentials are sent on every single request

Unlike token-based systems where credentials are exchanged once for a short-lived token, Basic Auth requires the actual username and password to be transmitted with every request. This multiplies the chances of the credentials being captured.

No expiration and no easy revocation

If credentials leak, the only way to invalidate them is to change the password — which immediately breaks every client that uses it. There is no built-in way to expire credentials automatically or revoke just one session.

No fine-grained permissions

Basic Auth answers a yes/no question: are these credentials valid? It carries no information

about what the user is allowed to do. A real system usually needs different permissions for different users or applications, and Basic Auth has no built-in way to express that.

Only safe over HTTPS

On plain HTTP, the credentials travel in essentially plain text and anyone on the same network can capture them. Basic Auth assumes the entire connection is already encrypted, which is an additional requirement the API itself does not enforce.

Stronger alternatives

JSON Web Tokens (JWT)

The client logs in once with their credentials and receives a signed token. That token is sent with every subsequent request instead of the password. Tokens can carry claims (who the user is, what they can do), they have a built-in expiration time, and the server can verify them without storing session state. If a token leaks, it can be made worthless by changing the signing key, and it expires on its own anyway.

OAuth 2.0

OAuth 2.0 is a full authorization framework rather than just an authentication scheme. It supports scopes (limiting what a token can do), refresh tokens (renewing access without re-entering credentials), and delegated access (letting one application act on behalf of a user in another application). It is the standard used by services like Google, GitHub, and most enterprise APIs.

API keys

Simpler than JWT or OAuth: each client gets a unique key that they send in a header. The server checks the key against a list of valid ones. Keys can be revoked individually, rotated periodically, and given different permission levels. Less powerful than OAuth but a clear improvement over Basic Auth for many use cases.

What we would do differently in production

For a real deployment of this API, we would replace Basic Auth with JWT for the user-facing endpoints. The login flow would issue a short-lived access token (15–60 minutes) plus a longer-lived refresh token. Every request would carry the access token in the Authorization header as a Bearer token, and the entire API would only be served over HTTPS so tokens cannot be intercepted in transit.

6. Conclusion

This project pulled together several pieces of building a usable API: parsing real-world data out of XML, exposing it through CRUD endpoints, protecting those endpoints with authentication, and measuring the efficiency of two different ways to look up records.

On the API side, all five required endpoints work and return the right status codes for both successful and failed requests. Basic Auth is in place, and the testing screenshots confirm that requests without valid credentials are rejected with 401 Unauthorized.

On the DSA side, dictionary lookup was measured at roughly 280 times faster than linear search on the parsed dataset. This matches what the theory predicts — hash tables turn lookups into a constant-time operation, while linear search has to scan the list. For other kinds of queries (range searches, prefix searches, on-disk data), other structures like binary search trees or B-trees would be the right choice.

Basic Auth was a good fit for a learning exercise but would be replaced with JWT or OAuth 2.0 in any production version of this API. The reflection above lays out the specific reasons why.